

Fundamentals of Accelerated Data Science

NVIDIA Workshop — Summary of Task 1, Task 2 & Task 3, plus Review Quiz

Task 1: Data Science Overview & Data Manipulation

This section introduces the fundamentals of data science and why acceleration matters for modern, large-scale workloads.

Key Topics Covered

- What data science is: a multidisciplinary field combining math/statistics, computer science, and business domain knowledge to extract insight from data.
- Four types of analysis: Descriptive (what happened), Diagnostic (why it happened), Predictive (what will happen), Prescriptive (what to do about it).
- Data science includes non-analytical (upstream) work — ETL, data collection, cleaning, and data engineering — which can consume roughly 40% of a data scientist's time.
- Why GPUs matter: GPUs use parallel processing to handle petabyte-scale data, massive models, and real-time performance needs (LLMs, fraud detection, recommenders, genomics).
- DataFrames: two-dimensional labeled data structures; data can be structured (tables, CSVs), semi-structured (JSON, graphs), or unstructured (text, images, audio).
- Vectorized operations: applying computation across whole columns at once (leveraging vector width and multiple cores) is far faster than row-by-row loops.
- RAPIDS cuDF: a GPU DataFrame library with a pandas-style API for loading, joining, filtering, and aggregating data; supports GroupBy (split-apply-combine), user-defined functions (UDFs via Numba JIT), and accelerated file readers.
- Apache Arrow: the columnar, cross-language in-memory format RAPIDS relies on — enables zero-copy data sharing across tools and efficient GPU vectorization.
- cudf.pandas: zero-code-change GPU acceleration for existing pandas code, with automatic CPU fallback — up to 150x faster than CPU-only pandas.
- Polars + RAPIDS cuDF: a Rust-based DataFrame library using lazy execution and a query optimizer; RAPIDS accelerates it via a GPU engine in the lazy API.
- Lazy execution: queries are built into an optimized execution plan rather than run immediately, enabling predicate/projection pushdown and out-of-core processing.
- Dask + RAPIDS: distributed parallel computing across multi-GPU systems for datasets larger than memory.
- Choosing tools by scale: pandas/Polars for smaller/medium data, Dask/Spark for massive, cluster-scale data.
- Visualization: cuxfilter (cross-filtering dashboards) and Datashader (rasterizing large datasets into images) bring GPU acceleration to data visualization.
- Interoperability: CuPy, DLPack, and the CUDA array interface let GPU libraries share array data without copying or converting it.

Task 2: Graph Analytics

This section covers graph theory fundamentals and how RAPIDS accelerates graph algorithms on the GPU.

Key Topics Covered

- Anatomy of a graph: nodes (vertices) and edges connecting them; edges can be directed, undirected, weighted, or unweighted.
- Degree and neighborhood: in-degree/out-degree count a node's connections; 1-hop and 2-hop neighborhoods describe nearby nodes.
- Common use cases: graphs model relationships across industries — social networks, fraud detection, recommendation systems, logistics, and more.
- Representing graphs: adjacency matrix (shows each node's neighborhood, works for weighted graphs too), degree matrix (counts connections per node), Laplacian matrix (shows graph 'smoothness'), and adjacency list (alternate compact representation).
- Single-Source Shortest Path (SSSP): computes the shortest path from one source node to all other reachable nodes in a weighted graph (no negative-weight cycles); used in road networks, logistics, and communications.
- Centrality: a family of metrics measuring how 'important' a node is within a network.
- nx-cugraph: a GPU-accelerated backend for NetworkX that bridges NetworkX's ease of use with high-performance GPU graph computation. It uses more efficient data structures than NetworkX's dictionary-based approach and minimizes CPU–GPU data movement.
- Three ways to enable nx-cugraph: (1) an environment variable (NX_CUGRAPH_AUTOCONFIG) for no-code acceleration, (2) an explicit backend keyword argument on supported NetworkX APIs, (3) type-based dispatching — importing the cugraph backend and instantiating its Graph type directly.

Task 3: Machine Learning

This section covers machine learning fundamentals, the end-to-end ML workflow, and how RAPIDS accelerates each stage.

Key Topics Covered

- ML vs. data science: data science has a broader scope (statistics, visualization, insight generation); machine learning focuses specifically on algorithms that learn from data to make predictions.
- Rules-based programming vs. learned models: traditional programs apply explicit rules to inputs to get outputs; ML instead learns a model from (input, output) examples via a learning algorithm.
- Training vs. inference: training fits a model to data; inference applies the trained model to new, unseen inputs to generate predictions.
- Supervised learning: trained on labeled data — Regression predicts a continuous value, Classification predicts a discrete label.
- Unsupervised learning: works on unlabeled data — Clustering groups similar points, Dimensionality Reduction compresses features while preserving information.
- Lifelong/continuous learning: comparing predictions to actual outcomes to detect data drift and retrain models as new data arrives.

- Common use cases by industry: spam detection, fraud detection, churn prediction, patient similarity, house price prediction, product recommendations, and more, across supervised and unsupervised tasks.
- ML workflow: Data Preparation → Feature Engineering → Model Selection/Training → Model Evaluation → Model Deployment, with continuous monitoring and retraining.
- Feature engineering: transforming raw data into a format that better represents the underlying problem — often the most impactful step for model performance; accelerated with cuDF.
- Classification algorithms compared: K-Nearest Neighbors (similarity-based, 'lazy learner'), Logistic Regression (efficient, less prone to overfitting, binary outcomes), Decision Trees (no feature scaling needed, robust to outliers, but prone to overfitting).
- Decision trees & ensembles: trees split data recursively to maximize purity but overfit easily; pruning and ensemble methods (combining multiple trees) help address this.
- Clustering algorithms compared: K-Means (fast, simple, assumes spherical clusters, needs k specified) vs. DBSCAN (density-based, finds arbitrary shapes, robust to outliers, automatically determines cluster count). Use cases: market segmentation, anomaly detection, data imputation.
- Model evaluation: balancing accuracy against inference speed/latency; using cross-validation to guard against overfitting; evaluating how performance scales with data volume.
- Model tuning: feature selection (finding the optimal feature subset) and hyperparameter tuning (e.g., grid search); cuML accelerates this training loop.
- XGBoost + RAPIDS: Extreme Gradient Boosting is a scalable, distributed gradient-boosted decision tree algorithm considered a 'gold standard' for classification/regression; RAPIDS integrates with it for GPU-accelerated data loading and training.
- Triton Inference Server: open-source, multi-framework (TensorFlow, PyTorch, ONNX) inference serving software supporting concurrent execution, dynamic batching, and model versioning — used to deploy models at scale.
- Full accelerated ML stack: cuDF (data) → cuML + XGBoost (training) → Triton (deployment), delivering higher productivity, more experimentation, and lower total cost of ownership.

Review Quiz

10 questions covering Task 1 (Data Manipulation), Task 2 (Graph Analytics), and Task 3 (Machine Learning). Answers are listed separately at the end.

1. What in-memory columnar data format does RAPIDS cuDF use internally for efficient data processing?

- A. Apache Arrow
- B. Apache Parquet
- C. Protocol Buffers
- D. JSON

2. Which RAPIDS feature lets you accelerate existing pandas code on the GPU with zero code changes?

- A. cudf.pandas
- B. Dask-cuDF
- C. cuxfilter
- D. nx-cugraph

3. Why are vectorized operations preferred over sequential loops in accelerated data science?

- A. They can leverage parallel hardware (GPU/SIMD) instead of processing one item at a time
- B. They use less disk storage
- C. They always produce more accurate results
- D. They require no memory

4. In graph theory, what does the 'degree' of a node refer to?

- A. The number of connections (edges) a node has
- B. The physical distance between two nodes
- C. The weight assigned to a node
- D. The number of graphs a node belongs to

5. What is nx-cugraph?

- A. A GPU-accelerated backend for NetworkX graph algorithms
- B. A visualization library for dashboards
- C. A replacement for pandas DataFrames
- D. A distributed computing scheduler

6. Which algorithm finds the shortest path from one designated node to all other reachable nodes in a weighted graph?

- A. Single-Source Shortest Path
- B. K-Means Clustering
- C. Centrality
- D. XGBoost

7. What is the key difference between supervised and unsupervised machine learning?

- A. Supervised learning uses labeled data; unsupervised learning finds patterns in unlabeled data
- B. Supervised learning is always faster to train
- C. Unsupervised learning requires a GPU while supervised does not
- D. Supervised learning only works on images

8. Which clustering algorithm can find clusters of arbitrary shape and doesn't require specifying the number of clusters in advance?

- A. DBSCAN
- B. K-Means
- C. Decision Tree
- D. Logistic Regression

9. What common weakness do decision trees have, and what technique helps address it?

- A. Overfitting; addressed by pruning or ensembling
- B. Underfitting; addressed by adding more features
- C. Slow inference; addressed by feature scaling
- D. High bias; addressed by increasing learning rate

10. What is the role of Triton Inference Server in the RAPIDS/ML pipeline described in Task 3?

- A. Deploying and serving trained AI models at scale in production
- B. Cleaning and transforming raw data before training
- C. Visualizing large graph datasets
- D. Accelerating pandas DataFrame operations

Answer Key

1. Correct Answer: A. Apache Arrow

Explanation: cuDF uses the Apache Arrow columnar memory format internally, enabling zero-copy sharing between tools and efficient vectorized/GPU processing.

2. Correct Answer: A. cudf.pandas

Explanation: cudf.pandas lets you write standard pandas code, then load the extension to run it on the GPU (with automatic CPU fallback) without rewriting anything.

3. Correct Answer: A. They can leverage parallel hardware (GPU/SIMD) instead of processing one item at a time

Explanation: Vectorized operations apply a computation across an entire column/array at once, taking advantage of vector width and multiple cores rather than iterating sequentially.

4. Correct Answer: A. The number of connections (edges) a node has

Explanation: Degree (in-degree/out-degree for directed graphs) counts the number of edges connected to a node; the degree matrix records this for every node.

5. Correct Answer: A. A GPU-accelerated backend for NetworkX graph algorithms

Explanation: nx-cugraph bridges NetworkX's ease of use with GPU-accelerated performance, offering three activation methods: environment variable, backend keyword argument, or type-based dispatching.

6. Correct Answer: A. Single-Source Shortest Path

Explanation: SSSP computes distances/predecessors from one source node to all reachable nodes — used in road networks, logistics, and communications.

7. Correct Answer: A. Supervised learning uses labeled data; unsupervised learning finds patterns in unlabeled data

Explanation: Supervised learning trains on labeled data to predict outcomes (classification/regression); unsupervised learning discovers hidden structure in unlabeled data (clustering/dimensionality reduction).

8. Correct Answer: A. DBSCAN

Explanation: DBSCAN clusters based on spatial density and can find arbitrarily shaped clusters while automatically determining the number of clusters, unlike K-Means which assumes spherical clusters and a fixed k.

9. Correct Answer: A. Overfitting; addressed by pruning or ensembling

Explanation: Decision trees are prone to overfitting the training data; pruning (trimming branches) and ensemble methods (combining multiple trees) reduce this risk.

10. Correct Answer: A. Deploying and serving trained AI models at scale in production

Explanation: Triton is an open-source inference serving platform supporting multiple frameworks (TensorFlow, PyTorch, ONNX), concurrent execution, and dynamic batching — the deployment stage of the ML workflow.